

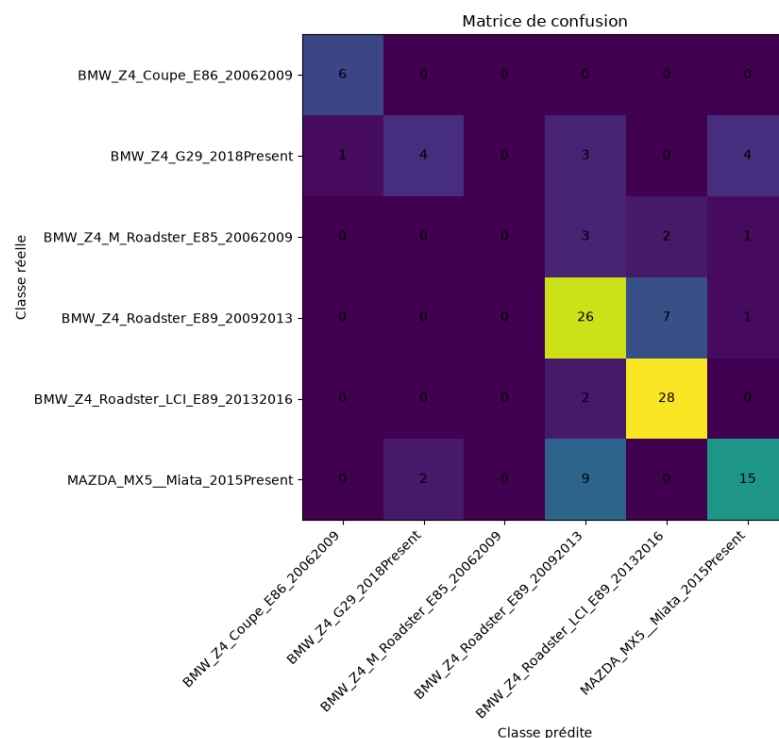
SAE 5.01

Rapport de semaine 02/10/2026

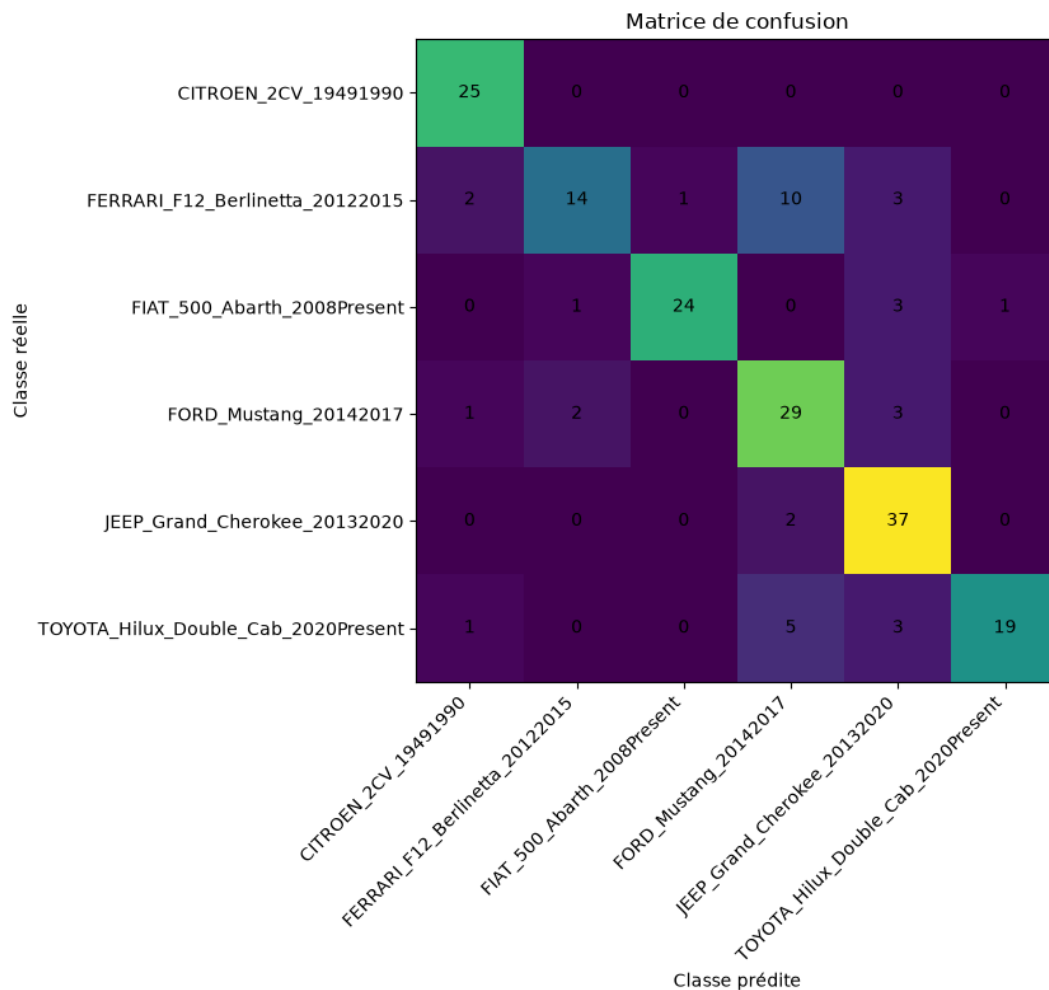
Avancement au niveau du modèle :

Cette semaine, nous avons corrigé un problème identifié sur le modèle de reconnaissance, il existait un écart important de précision entre les voitures disposant de peu de photos dans le dataset (une vingtaine) et celles qui en possédaient plus de cent. Afin de faire quelques vérifications concernant ce déséquilibre, nous avons sélectionné les voitures disposant du plus grand nombre de photos, en conservant 6 modèles suffisamment différents les uns des autres pour vérifier que le modèle parvenait réellement à les distinguer.

Avec ce nouveau dataset, le modèle obtient désormais une précision de 83,87 % sur les données de test. La matrice de confusion montre que la majorité des prédictions sont correctes (visible sur la diagonale), à l'exception de la Ferrari F12 et de la Ford Mustang, encore régulièrement confondues entre elles : les photos utilisées pour ces deux modèles montrent des véhicules rouges aux formes assez proches, ce qui complique leur différenciation pour le modèle.



Test avec des voitures qui se ressemblent avec un déséquilibre au niveau du dataset



Test avec les voitures ayant le + d'images dans le dataset et qui se ressemblent pas

Pour la semaine à venir nous aimerions changer la structure de ce code en ajoutant des classes et avoir une manière de coder plus "compréhensible". Après avoir établi une vraie base nous modifierons les poids de notre modèle pour avoir un moyen de comparaison entre les dataset et les voitures le plus efficace et précis possible.

Avancement au niveau de l'application :

Le développement du frontend s'est poursuivi cette semaine avec plusieurs évolutions. Les deux boutons permettant de prendre une photo ont été redessinés. Le bouton "Analyser" envoie désormais la photo vers le serveur lorsque l'utilisateur est connecté, dans le cas contraire, un popup "Image analysée" s'affiche avec l'image et un message invitant l'utilisateur à se connecter pour sauvegarder son analyse dans sa collection.

La page "Ma bibliothèque" a également été modifiée, les photos sont maintenant conservées sur le compte de l'utilisateur plutôt qu'en local sur l'appareil, avec un aperçu zoomable et un bouton permettant de les supprimer. Un emplacement a été laissé vide pour la légende de chaque voiture, qui sera rempli plus tard par le résultat du modèle de reconnaissance. Enfin, le token reçu lors de la connexion est désormais conservé, ce qui évite à l'utilisateur de devoir se reconnecter à chaque ouverture de l'application.

Difficultés rencontrées et solutions :

Le token de connexion n'était conservé nulle part donc on l'a stocké dans `AuthService.js`

Les photos ne devaient plus rester stockées en local maintenant elles transitent par le serveur

Une image supprimée restait affichée à la place de la suivante donc on a ajouté une clé unique par photo dans la liste

Traçabilité de l'usage de l'IA :

Pour la partie frontend, le développement a été réalisé majoritairement à la main, avec une assistance de GitHub Copilot pour l'autocomplétion de structures répétitives (widgets, boutons, containers), comme la semaine précédente.

La mise en place de la protection des routes et du système de sauvegarde des photos (`requireAuth.js`, `cars.js`) a été réalisée en mode hybride avec l'aide de Claude. Le point de départ était un problème concret, les routes de gestion des photos n'étaient protégées par aucune vérification, n'importe qui aurait pu consulter ou supprimer les photos d'un autre utilisateur. La structure proposée pour le middleware de vérification du token a été conservée telle quelle, car elle reprenait simplement le fonctionnement déjà utilisé pour la connexion. Pour les routes des voitures, la première proposition utilisait un stockage des images en fichiers séparés sur le serveur (via "multer"), cette partie a été écartée au profit d'un envoi en base64 directement dans le corps de la requête, plus cohérent avec le reste de notre API et plus simple à mettre en place. Les noms de champs et les limites de taille d'image ont été adaptés pour correspondre à notre modèle de données déjà en place côté application.

Voici les différents prompts effectués :

Prompt pour requireAuth.js

"j'ai fait les routes pour les voitures (upload, liste, suppression) mais la le probleme c'est que n'importe qui peut appeler ces routes meme sans etre connecté, du coup tout le monde pourrait voir ou supprimer les photos des autres utilisateurs. il me faudrait un truc qui bloque l'accès si y'a pas de token valide avant d'executer la route, un peu comme on a deja avec le JWT pour la connexion"

Prompt pour cars.js

"maintenant j'aimerais faire les routes pour sauvegarder une photo de voiture scannée côté utilisateur connecté, genre créer, lister les voitures, récupérer une image et supprimer, en m'inspirant de comment j'ai déjà fait auth.js, et en protégeant tout ça avec le middleware qu'on vient de faire"

Pour la partie du modèle il n'y a pas eu beaucoup de changement à part quelques tests réalisés pour vérifier la précision du modèle. Nous n'avons pas utilisé l'ia pour coder quoi que ce soit en plus pour cette partie.